CAMPUSBANK

Student Banking & Finance Management System

Project Report · Academic Simulation

October 2026

React · Express / Node.js · MySQL 8

1. Executive Summary

CampusBank evolves the existing Bank Management System into a student-oriented banking and personal finance simulation. It retains customer/student profiles, branches, accounts, deposits, withdrawals, transfers, transaction history, education loan records, and repayments. Student finance additions classify allowance/income and eligible expenses, report monthly activity, and track savings goals linked to accounts. All data is fictional; the application does not connect to real bank accounts or process real money.

2. Requirements and Existing Functionality

- Create and search student profiles; associate accounts with students and branches.
- Post deposits and withdrawals; transfer funds atomically; inspect derived balances and transaction history.
- Maintain branch and education loan records and linked payments.
- Categorize allowance, scholarship, family support, and other income; categorize Food, Travel, Hostel, Books, College Fees, Entertainment, and Other spending.
- Create, edit, list, and delete student savings goals; show progress based on a linked account balance.
- Provide account, branch, transaction, loan, student-month, and expense-category reports.

Application roles considered are administrator, teller/operator, read-only manager/auditor, and optional student. The project has no authentication or persisted role model, so it does not provide student-level access control.

3. Relational Model

The original six banking tables remain in place. Student profile attributes extend `customers` to avoid duplicating profile details and preserve existing account/loan foreign keys. Two new relations support the student features: `expense_categories` and `savings_goals`.

Table	Keys / relationship	Role
customers	customer_id PK; student_id UNIQUE	Student identity, contact, institution, course, graduation year
branches	branch_id PK; branch_code UNIQUE	Branch directory
accounts	customer_id FK; branch_id FK	Student account and opening balance
transactions	source/destination account FKs; optional expense_category_id FK	Deposit, withdrawal, transfer, and repayment ledger
loans	customer_id FK; branch_id FK	Education loan record and terms
loan_payments	loan_id FK; unique transaction_id FK	Payment details linked to repayment debit
expense_categories	category_type + category_name UNIQUE	Normalized income and expense labels
savings_goals	customer_id FK; account_id FK	Student target linked to an existing account

4. ER Relationships and Normalization

One student owns many accounts and may have many loans and savings goals. One branch has many accounts and loans. An account is the source of many transactions and may be the destination of transfers. A loan has many loan payments; each payment links to one ledger transaction. A finance category may classify many deposit or withdrawal records.

First normal form is met by atomic profile, account, transaction, payment, and goal attributes. Second normal form is met because each relation uses a single-column key and non-key attributes depend on that key. Third normal form is supported by keeping student details, branch facts, category names, loan facts, and payment facts in their appropriate relations rather than repeating those facts in ledger rows. Current balances are derived from opening balance and posted ledger movements through `AccountSummaryView` rather than independently edited totals.

5. Database and Financial Rules

- Primary/foreign keys, unique constraints, checks, and defaults enforce identity and valid monetary domains. Monetary columns use DECIMAL.
- `AccountSummaryView` derives current balance. `TransferFunds` demonstrates a database-side transfer. Triggers validate active accounts, repayment-to-ledger consistency, and category type.
- The Express financial operations use MySQL transactions and `SELECT ... FOR UPDATE`; transfers acquire locks in ascending account ID order and roll back on failure.
- Monthly income counts posted, categorized deposits. Categorized posted withdrawals count as expenses. Transfers are excluded from income and spending; loan repayments are reported separately.
- Savings progress is `LEAST(linked account balance, target amount)`, clamped in the UI to 0–100%. No funds are reserved or moved; multiple goals may refer to one balance.

6. Backend and API

The existing Express API and mysql2 promise pool remain. Customer CRUD is retained and student profile CRUD is also exposed under `/api/students`. Added routes expose finance categories, savings-goal CRUD, and `/api/reports/students/:id/monthly?month=YYYY-MM`. Deposit and withdrawal requests accept an optional finance category; API validation and a database trigger ensure income categories apply to deposits and expense categories to withdrawals. Values use prepared SQL parameters.

The monthly report returns available balance, categorized income, categorized expenses, separate loan repayments, and grouped expense totals. The savings-goal API verifies that the selected account belongs to the selected student. Authentication/authorization and production security controls are outside the academic scope.

7. Frontend

The React interface is branded CampusBank and includes the student dashboard, student profiles, account management, categorized transaction forms and filters, savings goals with progress bars, education loan/payment views, and reports. Monthly dashboard selectors allow a student and month to be chosen. The transaction history supports student, category, type, status, and month filters. The UI explains that goals do not reserve balances and that transfers are not spending.

8. Database Setup and Migration

For a fresh database, execute the root `bank_management.sql` once against an empty `bank_management` schema. It now preserves a pre-existing database instead of dropping it; use it only for first-time schema creation. For a database from the prior project, back up and apply `migrations/001_campusbank_student_finance.sql` once. The migration is additive and leaves existing ledger and account records intact; it backfills only the known fictional seed profiles.

9. Verification Results

Check	Result	Coverage / boundary
Backend tests	15 passed	CRUD, validation, parameterized filters, safe transfer lock ordering/rollback, categories, student profiles/reports, goal ownership.
Frontend production build	Passed	Existing Rollup build completed.
SQL execution against MySQL	Pending	No valid database credentials available in this environment; schema/migration were reviewed but not executed live.
Frontend-to-MySQL E2E	Pending	Requires configured database credentials and a live demonstration run.

The tests use mocked database pools/connections. They do not prove live MySQL syntax or integration. Before presenting a live demonstration, execute the fresh schema or migration, configure backend/.env, verify `/api/health`, and capture actual SQL result grids.

10. Demonstration and Viva

Suggested flow: create or edit a student profile; inspect linked accounts; post a categorized allowance deposit and expense; review monthly totals and category filters; transfer between accounts and verify exclusion from income/spending; create a linked savings goal; review education loan payments; attempt an overdrawn withdrawal/transfer and an invalid account, then verify balances and ledger rows remain unchanged.

- 3NF: each fact is recorded once in its appropriate relation; category labels are lookup data and balance is derived from the ledger.
- Atomic transfer: row locks, account/status/funds checks, one transaction, commit on success, rollback on error.
- Savings goal: balance-based estimate capped at target; does not earmark funds.
- Loan model: tracks principal and payments but does not implement approval, real disbursement, or an amortization engine.
- Limitations: no real payment rails, authentication, student isolation, regulatory workflows, or production hardening.

Appendix — Project Files

- `bank_management.sql` — fresh schema, seed data, database objects, demo and report queries.
- `migrations/001_campusbank_student_finance.sql` — additive upgrade from the original database.
- `PHASE1_DATABASE_DESIGN.md` — database design and normalization background.
- `backend/README.md`, `frontend/README.md`, `SUBMISSION_GUIDE.md` — setup and use instructions.